- `ConsumesAttribute`—Can be used to restrict the allowed formats an action method can accept. If your action is decorated with `[Consumes("application/json")]`, but the client sends the request as Extensible Markup Language (XML), the resource filter will short-circuit the pipeline and return a `415 Unsupported Media Type` response.
- `SkipStatusCodePagesAttribute`—This filter prevents the `StatusCodePagesMiddleware` from running for the response. This can be useful if, for example, you have both web API controllers and Razor Pages in the same application. You can apply this attribute to the controllers to ensure that API error responses are passed untouched, but all error responses from Razor Pages are handled by the middleware.

Resource filters are useful when you want to ensure that the filter runs early in the pipeline, before model binding. They provide an early hook into the pipeline for your logic so you can quickly short-circuit the request if you need to.

Look back at listing 22.1 and see whether you can refactor any of the code into a resource filter. One candidate line appears at the start of both the `Get` and `Edit` methods:

```
if (!IsEnabled) { return BadRequest(); }
```

This line of code is a *feature toggle* that you can use to disable the availability of the whole API, based on the `IsEnabled` field. In practice, you'd probably load the `IsEnabled` field from a database or